

Chapitre 1 : K Plus Proches Voisins (KNN)

Apprentissage Supervisé

Dr Seydou Nourou SYLLA

Université Alioune DIOP de Bambey
Master SI/SR

2 mars 2026

Plan du chapitre

- 1 Intuition et principe
- 2 Algorithme from scratch
- 3 Choix de k et complexité
- 4 Forces, faiblesses et cas d'usage

L'intuition derrière KNN

Principe de base :

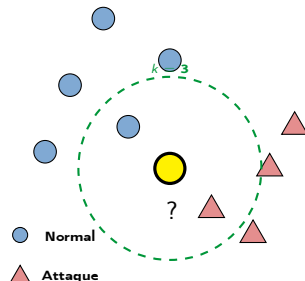
"Dis-moi qui sont tes voisins, je te dirai qui tu es."

Idée :

- Pour prédire y d'un nouveau point x , chercher les k exemples **les plus proches** dans $\mathcal{D}_{train}$
- **Classification** : voter (majorité)
- **Régression** : moyenner

Aucun paramètre appris pendant l'entraînement !

⇒ Algorithme **paresseux** (*lazy learning*)



Exemple d'application 1 – Interprétation KNN

Question : Cette connexion est-elle une intrusion ?

$$x_{new} = \{\text{duration} = 0, \text{src_bytes} = 0, \text{dst_bytes} = 0, \text{logged_in} = 0, \dots\}$$

Démarche KNN :

- 1 Calculer la distance entre x_{new} et **toutes** les connexions du train
- 2 Garder les $k = 5$ connexions les plus proches
- 3 Si parmi elles 4 sont des attaques → **prédiction : attaque**

Voisin	distance	src_bytes	logged_in	label
voisin 1	0.12	0	0	attack
voisin 2	0.23	0	0	attack
voisin 3	0.31	105	0	normal
voisin 4	0.35	0	0	attack
voisin 5	0.40	0	0	attack
Vote majoritaire :				attack (4/5)

Plusieurs distances possibles :

Nom	Formule	Usage
Euclidienne	$\sqrt{\sum_j (x_j - x'_j)^2}$	Variables continues
Manhattan	$\sum_j x_j - x'_j $	Données éparses
Minkowski	$\left(\sum_j x_j - x'_j ^p\right)^{1/p}$	Généralisation
Chebyshev	$\max_j x_j - x'_j $	Jeux, robotique

Remarque

La qualité de KNN dépend fortement du choix de la distance et de la normalisation des données.

Distance euclidienne et normalisation

Distance euclidienne entre deux points x et x' :

$$d(x, x') = \|x - x'\|_2 = \sqrt{\sum_{j=1}^p (x_j - x'_j)^2}$$

Problème sans normalisation :

- `src_bytes` : de 0 à 1 000 000
- `logged_in` : $\{0, 1\}$
- La feature à grande échelle **domine** la distance !

Standardisation (Z-score) :

$$x'_j = \frac{x_j - \mu_j}{\sigma_j}$$

où μ_j et σ_j sont la moyenne et l'écart-type de la feature j sur le train.

⇒ Toutes les features contribuent équitablement à la distance.

Algorithme 1 : K Plus Proches Voisins (KNN)

Input : $\mathcal{D}_{train} = \{(x_i, y_i)\}_{i=1}^n$, nouveau point x_{new} , entier k

Output : Prédiction $\hat{y}$

Phase d'entraînement ;

Mémoriser $\mathcal{D}_{train}$ (aucun calcul !);

Phase de prédiction ;

Calculer $d_i = d(x_{new}, x_i)$ pour tout $i \in \{1, \dots, n\}$;

Trier les distances : obtenir $d_{(1)} \leq d_{(2)} \leq \dots \leq d_{(n)}$;

$\mathcal{N}_k \leftarrow$ indices des k plus petites distances;

Classification : $\hat{y} = \text{mode}(\{y_i : i \in \mathcal{N}_k\})$;

Régression : $\hat{y} = \frac{1}{k} \sum_{i \in \mathcal{N}_k} y_i$;

Impact du paramètre k

k trop petit ($k = 1$) :

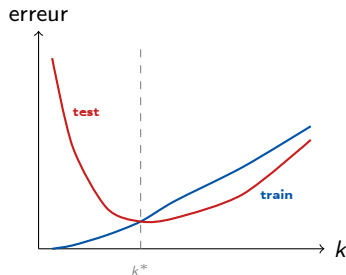
- S'adapte parfaitement au train
- Très sensible au bruit
- **Overfitting** (sur-apprentissage)

k trop grand ($k = n$) :

- Prédit toujours la classe majoritaire
- Ignore la structure locale
- **Underfitting** (sous-apprentissage)

Bon k :

- Typiquement impair (évite les ex-aequo)
- Choisi par **validation croisée**
- Souvent $k = \sqrt{n}$ comme point de départ



Phase	Temps	Espace
Entraînement	$O(1)$	$O(n \cdot p)$
Prédiction (1 exemple)	$O(n \cdot p)$	$O(n)$
Prédiction (m exemples)	$O(m \cdot n \cdot p)$	$O(n)$

Problème de passage à l'échelle :

- Dans l'exemple d'application 1 : $n = 100\,778$, $p = 8$
- Prédire 1 exemple : $\approx 800\,000$ opérations
- Prédire 25 000 exemples : ≈ 20 milliards d'opérations !

Limite critique

KNN est **inadapté** aux datasets volumineux ou haute dimension sans indexation spatiale.

Solutions :

- **KD-Tree** : $O(p \log n)$ en moyenne
- **Ball Tree** : efficace en haute dimension
- **LSH** (Locality Sensitive Hashing)

Règle pratique

Pour $n > 50\,000$, utiliser `algorithm='ball_tree'` ou `'kd_tree'` dans sklearn.

Forces et faiblesses de KNN

Forces :

- + Simple à comprendre et implémenter
- + Aucun entraînement requis
- + Naturellement multi-classes
- + S'adapte aux distributions complexes
- + Sensible aux nouvelles données (mise à jour instantanée)

Faiblesses :

- Lent à l'inférence ($O(n)$ par prédiction)
- Sensible à la normalisation
- Malédiction de la dimensionnalité
- Sensible aux valeurs aberrantes
- Stockage de tout le dataset

Contexte (ex. app. 1)	Verdict
Dataset statique, batch prediction	OK
Détection temps réel (streaming)	Problématique
Features très hétérogènes	Normalisation indispensable
Nouvelles données (évolution)	Bonne adaptabilité

Idée : Les voisins plus proches méritent un poids plus grand.

Vote pondéré :

$$\hat{y} = \arg \max_c \sum_{i \in \mathcal{N}_k} w_i \cdot 1[y_i = c] \quad \text{avec} \quad w_i = \frac{1}{d(x_{new}, x_i) + \varepsilon}$$

À retenir

- KNN est un algorithme **lazy** : entraînement $O(1)$, prédiction $O(n)$
- Le paramètre k contrôle le compromis biais/variance
- La **normalisation** est indispensable
- Performant sur l'exemple d'application 1 ($> 99\%$ d'accuracy) mais lent à l'échelle

Hyperparamètre	Valeur recommandée (ex. app. 1)
k (nombre de voisins)	3 ou 5 (validation croisée)
Métrique de distance	Euclidienne (données normalisées)
Pondération	Distance (légèrement meilleur)